

IndiSign: Indian Sign Language Recognition Using Deep Learning

Abstract

This project presents a deep learning-based approach for recognizing Indian Sign Language (ISL) hand gestures from images. The system uses computer vision and Convolutional Neural Networks (CNNs) to learn visual patterns from Indian Sign Language images and classify them into their corresponding gesture categories.

The developed system demonstrates how deep learning can be applied to image-based sign language recognition and provides a foundation for developing real-time Indian Sign Language recognition applications.

1. Introduction

Indian Sign Language (ISL) is an important form of communication used by the deaf and hard-of-hearing community. Communication can become difficult when people who do not understand sign language interact with sign language users.

The project focuses on developing an image-classification pipeline that includes dataset preparation, preprocessing, model training, evaluation, and prediction.

2. Problem Statement

Communication between sign language users and people unfamiliar with sign language can be challenging.

The objective of this project is to develop a deep learning-based image classification system that can recognize Indian Sign Language gestures.

The system takes:

Input:

- Image containing an Indian Sign Language gesture

Output:

- Predicted Indian Sign Language class

The project aims to demonstrate how CNN-based deep learning can be used to automatically learn visual patterns associated with different hand gestures.

3. Dataset Description

The project uses an image dataset containing different Indian Sign Language gesture classes.

The dataset is organized according to gesture categories, allowing TensorFlow's image dataset utilities to automatically load the images and assign class labels.

The images are loaded using TensorFlow/Keras and resized to:

$224 \times 224 \times 3$

where:

- 224 represents image height.
- 224 represents image width.
- 3 represents the RGB color channels.

The dataset is processed in batches with a batch size of 32.

The repository contains the dataset under the `data/` directory.

4. Data Preprocessing

Data preprocessing is an important step in preparing image data for deep learning.

The following preprocessing operations are performed:.

- Resizing images to 224×224 pixels.

```
dataset = tf.keras.utils.image_dataset_from_directory(  
    'data',  
    image_size=(224,224),  
    batch_size = 32,  
    shuffle = True  
)  
  
Found 4972 files belonging to 24 classes.
```

- Preparing image-label pairs.

```
#Shape of the image and label  
for images,label in dataset.take(1):  
    print('Image Shape :',images.shape)  
    print('Label Shape :',label.shape)  
  
Image Shape : (32, 224, 224, 3)  
Label Shape : (32,)
```

- Preparing the dataset for model training and validation.
- Applying appropriate preprocessing before model prediction.

```
dataset = dataset.map(lambda x,y : ((x/255,y)))

scaled_iter = dataset.as_numpy_iterator()
batch = scaled_iter.next()
print(batch[0].min())
print(batch[0].max())

0.0
1.0
```

The dataset is loaded using TensorFlow's `image_dataset_from_directory()` functionality.

The resulting dataset contains image tensors and their corresponding integer class labels.

5. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to understand the structure and characteristics of the image dataset before training the model.

The analysis includes:

- Inspecting dataset dimensions
- Examining image shapes.

```
#Shape of the image and label
for images,label in dataset.take(1):
    print('Image Shape :',images.shape)
    print('Label Shape :',label.shape)

Image Shape : (32, 224, 224, 3)
Label Shape : (32,)
```

- Identifying class labels.

```
dataset_class = dataset.class_names

print(dataset_class)

['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y']
```

- Counting images belonging to different classes.

```
from collections import Counter

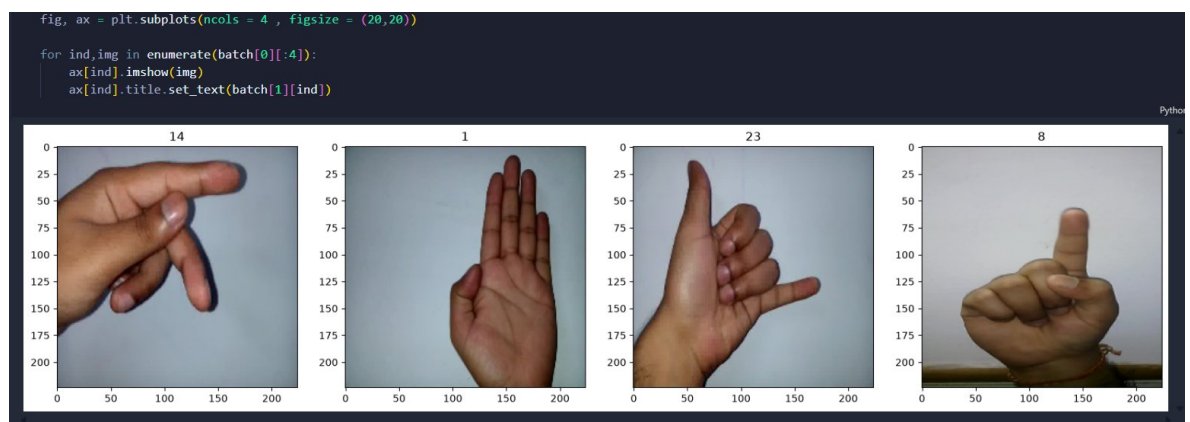
count = Counter()

for image,label in dataset:
    count.update(label.numpy())

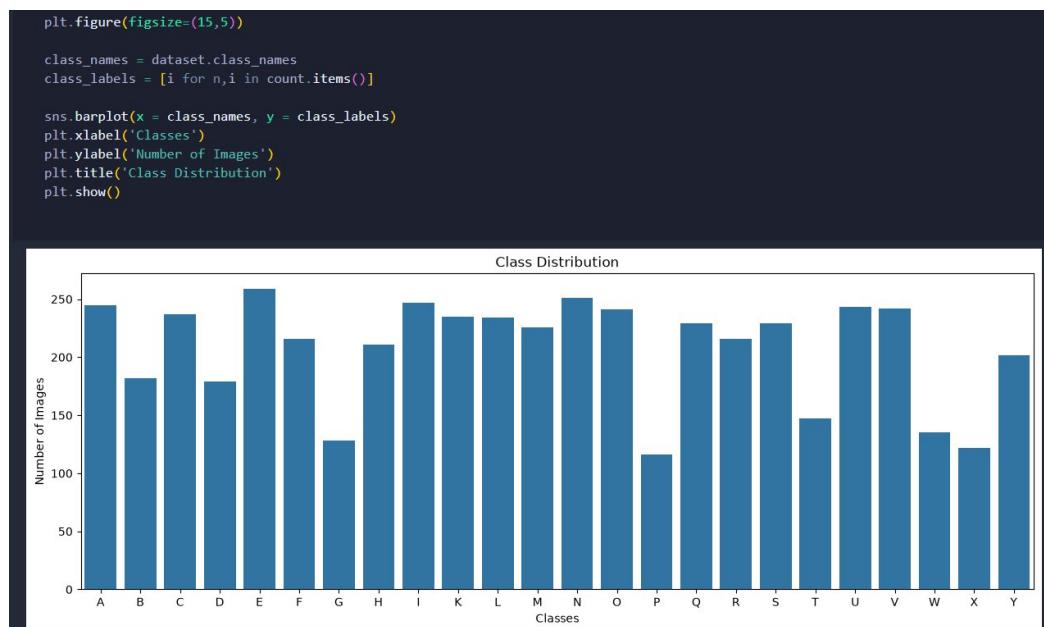
for class_names,counts in sorted(count.items()):
    print(dataset.class_names[class_names],':',counts)

A : 242
```

- Visualizing sample images.



- Studying class distribution.



Sample images were displayed using Matplotlib to understand the appearance and variation of the Indian Sign Language gestures.

Dataset visualization is useful because it helps identify potential problems such as incorrect images, inconsistent image sizes, or class imbalance.

6. Data Preparation

The dataset contains elements in the following form:

```
(images, labels)
```

The dataset is also prepared for training and validation so that the model can be trained on one portion of the data and evaluated on unseen validation samples.

```
train_size = int(len(dataset)*0.70)
val_size = int(len(dataset)*0.15)
test_size = (len(dataset) - train_size) - val_size

print("The Size of Train Size :",train_size)
print("The Size of Validation Size :",val_size)
print("The Size of Test Size:",test_size)

The Size of Train Size : 109
The Size of Validation Size : 23
The Size of Test Size: 24

train = dataset.take(train_size)
val = dataset.skip(train_size).take(val_size)
test = dataset.skip(train_size + val_size).take(test_size)
```

7. Data Augmentation

Data augmentation is used in image classification to create variations of training images and improve the model's ability to generalize.

Possible augmentation techniques include:

- Rotation
- Zoom
- Other suitable image transformations

However, augmentation must be selected carefully for sign language recognition. Some transformations, such as horizontal flipping, may change the meaning or orientation of a gesture.

The purpose of augmentation is to expose the model to variations in hand position and appearance while preserving the underlying class.

```
#Data Augmentation

data_augmentation = Sequential([
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomZoom(0.1),
    tf.keras.layers.RandomTranslation(0.1,0.1)
])
```

8. CNN Model Development

A **Convolutional Neural Network (CNN)** is used for Indian Sign Language image classification.

CNNs are particularly effective for image recognition because convolutional layers can automatically learn spatial features from images.

The model learns features progressively:

```
model = Sequential([

    tf.keras.layers.Input(shape=(224,224,3)),

    data_augmentation,

    #Block 1
    tf.keras.layers.Conv2D(32,(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(),

    #Block 2
    tf.keras.layers.Conv2D(64,(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(),

    #Block 3
    tf.keras.layers.Conv2D(128,(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Flatten(),

    tf.keras.layers.Dense(128,activation='relu'),

    tf.keras.layers.Dense(num_classes,activation='softmax')
])

model.summary()
```

This allows the network to learn useful visual representations directly from the training images.

9. CNN Model Architecture

The CNN architecture consists of multiple layers designed to extract features and classify the input gesture.

The general structure includes:

Layer (type)	Output Shape	Param #
sequential_2 (Sequential)	(None, 224, 224, 3)	0
conv2d_3 (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_4 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_5 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten_1 (Flatten)	(None, 86528)	0
dense_2 (Dense)	(None, 128)	11,075,712
dense_3 (Dense)	(None, 24)	3,096

Total params: 11,172,056 (42.62 MB)

Trainable params: 11,172,056 (42.62 MB)

Non-trainable params: 0 (0.00 B)

10. Model Training

The CNN model was trained using the prepared image dataset. The model was trained for multiple epochs, during which it learned patterns from the training images and updated its parameters based on the prediction error.

The training configuration includes:

- **Optimizer:** Adam
- **Loss Function:** Sparse Categorical Crossentropy
- **Metric:** Accuracy
- **Input Size:** $224 \times 224 \times 3$
- **Batch Size:** 32
- **Callback:** EarlyStopping

EarlyStopping was used to monitor the validation performance during training. Training was stopped automatically when the validation performance stopped improving, helping to reduce overfitting and unnecessary training.

Training logs were also recorded to monitor the model's performance throughout the training process. The following metrics were tracked:

```
model.compile(  
    optimizer = 'adam',  
    loss = tf.losses.sparse_categorical_crossentropy,  
    metrics = ['accuracy']  
)  
  
early_stopping = tf.keras.callbacks.EarlyStopping(patience=3,monitor='val_loss',restore_best_weights=True)  
  
log_dir = 'logs'  
tensorboard_callback = tf.keras.callbacks.TensorBoard(log_dir=log_dir)
```

- Training accuracy
- Validation accuracy
- Training loss
- Validation loss

```
hist = model.fit(  
    train,  
    validation_data = val,  
    epochs = 20,  
    callbacks = [early_stopping,tensorboard_callback]  
)
```

These training logs were later used to generate accuracy and loss graphs for analyzing the model's learning performance.

Final training results:

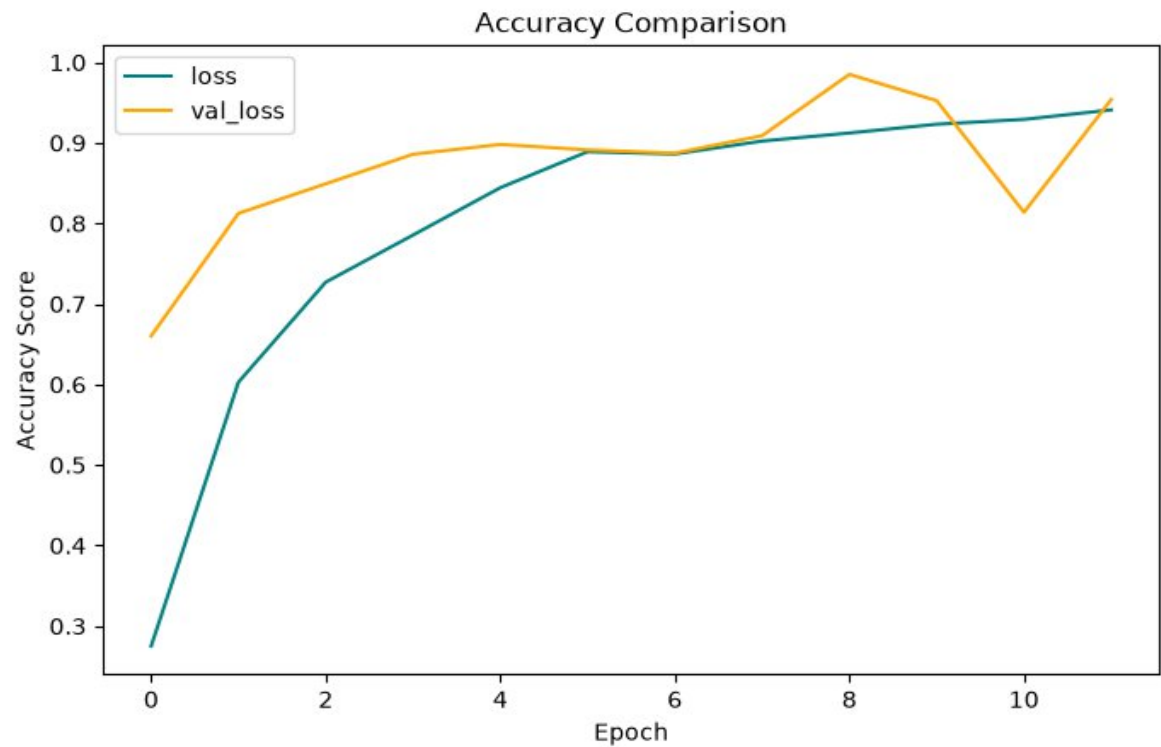
- **Training Accuracy:** 94.09%
- **Validation Accuracy:** 95.38%
- **Training Loss:** 0.1775
- **Validation Loss:** 0.1009

11. Model Training Performance

The training performance was analyzed using the recorded training logs. Accuracy and loss were plotted for the training and validation datasets to observe the model's learning behavior.

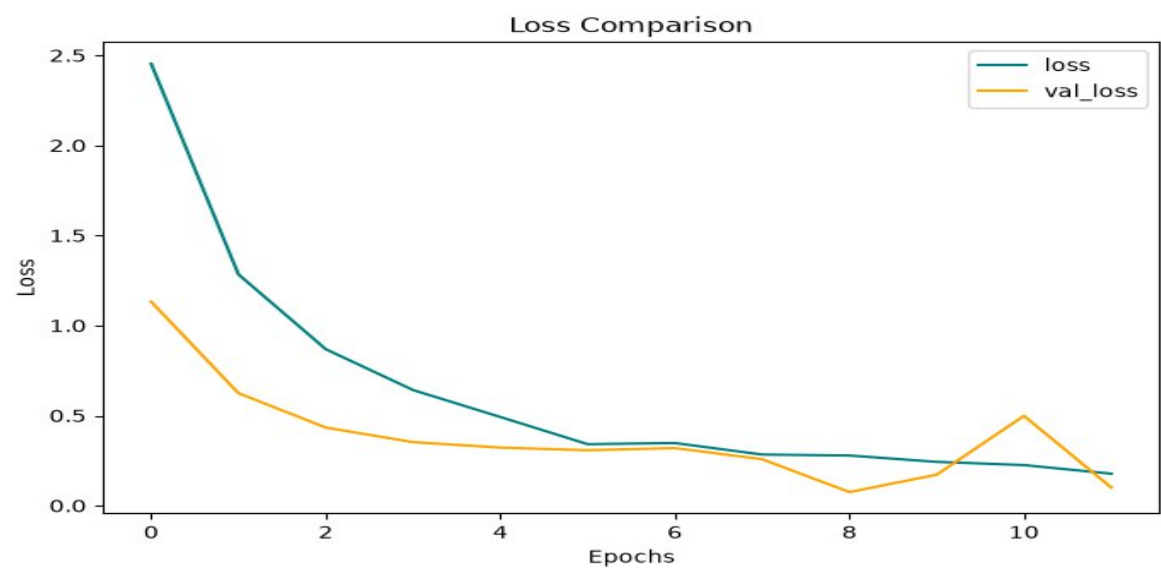
11.1 Accuracy Comparison

The accuracy graph compares the training and validation accuracy across the training epochs.



11.2 Loss Comparison

The loss graph compares the training and validation loss across the training epochs.



This is better for your report because you're **showing the actual code, logs, and graphs**, rather than making unsupported claims about the exact result

12. Model Evaluation

After training, the model is evaluated using unseen data to determine its ability to generalize beyond the training samples.

Model evaluation is important because high training performance alone does not guarantee good performance on new images.

The evaluation process can include:

- Accuracy
- Loss
- Prediction analysis
- Class-wise performance
- Confusion matrix

```
from sklearn.metrics import classification_report, confusion_matrix

print(classification_report(y_true, y_pred, target_names=dataset_class))
```

	precision	recall	f1-score	support
A	1.00	1.00	1.00	39
B	0.98	0.96	0.97	48
C	0.91	1.00	0.96	32
D	1.00	1.00	1.00	22
E	1.00	1.00	1.00	40
F	0.95	1.00	0.97	36
G	1.00	1.00	1.00	40
H	1.00	1.00	1.00	16
I	1.00	1.00	1.00	26
K	0.97	1.00	0.99	38
L	0.93	1.00	0.96	27
M	1.00	1.00	1.00	29
N	0.94	0.97	0.96	35
O	1.00	0.94	0.97	36
P	0.96	1.00	0.98	23
Q	1.00	0.97	0.99	36
R	1.00	0.97	0.99	34
S	1.00	1.00	1.00	21
T	0.97	0.97	0.97	40
U	1.00	1.00	1.00	20
V	0.86	0.90	0.88	21
W	0.92	0.73	0.81	15
X	1.00	0.95	0.97	41
...				
accuracy			0.98	748
macro avg	0.98	0.97	0.97	748
weighted avg	0.98	0.98	0.98	748

The results provide an indication of how effectively the CNN identifies Indian Sign Language gestures.

Note: The current repository README does not provide a final numerical test accuracy, so a specific accuracy value should not be inserted into the report unless it is taken directly from your notebook's final evaluation output.

13. Prediction System

After training, the model can be used to predict the class of a new Indian Sign Language image.

```
def prediction_system(image_path):
    img = cv2.imread(image_path)

    if img is None:
        print('Image Not found,',image_path)
        return

    img_rgb = cv2.cvtColor(img,cv2.COLOR_BGR2RGB)

    img_resized = cv2.resize(img_rgb,(224,224))

    img_batch = np.expand_dims(img_resized,axis=0)

    prediction = model.predict(img_batch,verbose=0)

    pred_index =np.argmax(prediction[0])

    print(prediction[0])
    confidence = prediction[0][pred_index]* 100

    class_name = dataset_class[pred_index]

    plt.figure(figsize=(5,5))
    plt.imshow(img_rgb)
    plt.title(f'Prediction :{class_name}\n'f'Confidence :{confidence:.2f}%')

    plt.axis('off')
    plt.show()

    return confidence,pred_clas
```

The predicted class is obtained from the model's output probabilities.

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
```



The class with the highest predicted probability is selected as the final prediction.

14. Technologies Used

The project uses the following technologies:

Technology	Purpose
Python	Programming
TensorFlow	Deep Learning
Keras	Neural Network Development
NumPy	Numerical Processing
Pandas	Data Processing
Matplotlib	Data Visualization
Scikit-learn	Supporting ML utilities
OpenCV	Image Processing
Jupyter Notebook	Development and experimentation
Git/GitHub	Version Control

The repository's current README also identifies TensorFlow/Keras, CNN, NumPy, Pandas, Matplotlib, Scikit-learn, OpenCV and Jupyter Notebook among the project's technologies.

15. Challenges Faced

Several challenges can arise while developing an image-based sign language recognition system.

13.1 Dataset Variations

Hand gestures may vary depending on:

- Hand position
- Orientation
- Lighting
- Background
- Individual hand shape

15.2 Class Distribution

Different gesture classes may contain different numbers of images. Such differences can affect model learning and should be monitored during dataset analysis.

15.3 Overfitting

Deep learning models can memorize training examples instead of learning general patterns. Techniques such as dropout, validation, and appropriate augmentation can help address this issue.

15.4 Computational Requirements

CNN training can require significant computational resources, especially when working with large image datasets and high-resolution input images.

16. Conclusion

This project developed a deep learning-based approach for **Indian Sign Language image recognition** using Convolutional Neural Networks.

The project covered the complete machine learning workflow, including dataset loading, image preprocessing, dataset exploration, visualization, model development, training, evaluation, and prediction.

The CNN learns visual characteristics from Indian Sign Language images and uses these learned representations to classify gesture images.

The project demonstrates the potential of Computer Vision and Deep Learning for sign language recognition and provides a foundation for future development of real-time gesture recognition and communication-support systems.

17. Reproducibility and Source Code

The complete project implementation is maintained in a GitHub repository. The repository contains the dataset directory, Jupyter Notebook implementation, requirements file, training performance graphs, and prediction output.

GitHub Repository:

[IndiSign – Indian Sign Language Prediction](#)

The main implementation is available in:

```
main.ipynb
```

and the required Python dependencies are provided in:

```
requirements.txt
```